Semantic Odyssey: Navigating Word Similarity Graphs

Alexander Davydenko Sophie Shu Abhirve Munipalle

CSC111 Project 2 — Phase 1 Proposal
March 2026

Problem Description and Project Question

Computational linguistics has long been interested in measuring how closely two words relate in meaning, and word embeddings offer one of the most practical ways to do so.

GloVe (Global Vectors for Word Representation), developed by Pennington et al. at Stanford [1], assigns each word a vector in high-dimensional space; words that tend to appear in similar linguistic contexts are placed near each other in that space. The *cosine similarity* between two word vectors gives a principled, numerical measure of their semantic closeness, ranging from -1 (opposites) to $+1$ (identical context). This makes it straightforward to build a graph: vertices are words, and edges connect pairs whose cosine similarity exceeds a chosen threshold, weighted by that similarity score.

Games like Semantle and Wordle have shown that word-meaning puzzles appeal to a broad audience. Inspired by these, we are designing *Semantic Odyssey*: an interactive word-navigation puzzle. The player starts at a given word and tries to reach a hidden target by hopping through a semantic similarity graph, choosing among nearby words at each step, in as few moves as possible. The player receives feedback on how *close* or *distant* each available word is, and the game will compute a provably achievable optimal path using breadth-first search on the same graph the player navigates.

Our project question is: Can a semantic similarity graph built from GloVe word embeddings serve as the foundation for an engaging word-navigation puzzle, with BFS providing a meaningful optimal-path benchmark for players?

A key design choice is that the graph is constructed lazily from a 400,000-word embedding model, so the project is computationally rich and grounded in real linguistic data without requiring expensive precomputation.

Computational Plan

Data

We use the GloVe `wiki-gigaword-50` model, which contains 50-dimensional vectors for 400,000 English words, trained on roughly 6 billion tokens from Wikipedia 2014 and the Gigaword 5 corpus [1]. We access the dataset through the `gensim` library's model downloader (`gensim.downloader.load`), which fetches and caches the vectors automatically. For example, the five words closest to "fire" by cosine similarity are "fires" (0.82), "wildfire" (0.80), "blaze" (0.79), "flames" (0.77), and "burning" (0.75), illustrating the rich semantic neighbourhood structure the vectors encode.

Graph Representation

We model the semantic word network as a directed, weighted, *implicit* graph $G = (V, E)$ where V is the GloVe vocabulary. Rather than materializing all $|V|^2 \approx 1.6 \times 10^{11}$ possible edges, we construct the graph lazily: for any vertex $w \in V$, its out-neighbourhood $N(w)$ is computed on demand by querying `model.most_similar(w, topn=50)`, returning the 50 words with the highest cosine similarity to w.

From these 50 candidates we apply a *fixed-bucket selection* strategy to ensure players always see a diverse spread of options:

- 3 *close* words with cosine similarity ≥ 0.65,
- 4 *related* words with similarity in $[0.45, 0.65)$,
- 3 *distant* words with similarity < 0.45,

giving each vertex a fixed out-degree of 10. If a bucket contains fewer than its quota the shortfall is filled from the remaining candidates in descending similarity order. We cache each computed neighbourhood in a dictionary so that no GloVe query runs more than once per session.

Computations

Breadth-first search (BFS). Our core computation is BFS on the implicit graph. Given a start word s and a target word t, BFS expands nodes level by level—each node's successors being its pre-computed out-neighbourhood—and returns the shortest path from s to t in terms of number of steps. Because the neighbour list is deterministic and fixed, the BFS path is provably achievable by a human player following the same edges, so the optimal path shown to the player is always one they could actually have taken.

Puzzle generation. Puzzle generation applies BFS with a fixed daily seed to identify suitable start-target pairs. We prefer pairs whose shortest BFS path is between 4 and 8 steps, balancing challenge and solvability. During loading, we run a quick BFS without a fixed target to validate candidate pairs, sharing the neighbour cache across calls. The full target-aware optimal path is then computed in a background thread while the player plays.

Target guarantee. At each player step we compute the cosine similarity between the current word and the target. If this similarity is ≥ 0.60—a threshold that captures morphological variants (e.g. "moons" $\rightarrow$ "moon," "roses" $\rightarrow$ "rose") while filtering out pairs that are only loosely related—the target is inserted into the 10-word neighbour list regardless of bucket placement, ensuring the player can always take the final step once they are genuinely close in semantic space.

Path comparison. On completing the puzzle the player's step count is compared against the BFS optimal, and the full paths are rendered side by side so the player can study where they diverged.

Visualisation and Interaction

The program is a Pygame application with four screens managed by a state machine. The first is a *loading screen* with an animated progress bar that reflects staged progress through vector loading and puzzle generation. Next is a *start screen* showing the target word and three selectable starting words. The *game screen* shows the current word, the player's path so far, and a 2×5 grid of clickable word tiles colour-coded by similarity label (*close*, *related*, *distant*). Finally, a *result screen* displays the player's path against the BFS-optimal path with step counts. An undo button allows

players to retrace steps. After winning, the player may continue to a new randomly chosen target from their current word.

New Library: `gensim`

We use the `gensim` library (version 4.x) [3] specifically for its pre-trained word vector interface, which we have not used in CSC110 or CSC111. Key components we rely on are:

- `gensim.downloader.load('glove-wiki-gigaword-50')` — fetches and caches the 400,000-word embedding model;
- `KeyedVectors.most_similar(word, topn=k)` — retrieves the k nearest neighbours of a word by cosine similarity in $O(|V| \cdot d)$ time ($d = 50$), which is the engine of our implicit graph construction;
- `KeyedVectors.similarity(w1, w2)` — computes pairwise cosine similarity in $O(d)$ time, used for the target guarantee check at every player step.

Computing cosine similarities across 400,000 words from scratch would be impractical within the project's timeline, so gensim's optimized vector operations are essential. Crucially, gensim handles only the embedding lookups; all graph construction, traversal, puzzle logic, and rendering are our own code.

References

References

[1] J. Pennington, R. Socher, and C. D. Manning, "GloVe: Global Vectors for Word Representation," in *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Doha, Qatar, 2014, pp. 1532–1543. [Online]. Available: https://nlp.stanford.edu/pubs/glove.pdf

[2] Stanford NLP Group, "GloVe: Global Vectors for Word Representation — Pre-trained vectors," 2014. [Online]. Available: https://nlp.stanford.edu/projects/glove/

[3] R. Řehůřek and P. Sojka, "Software Framework for Topic Modelling with Large Corpora," in *Proceedings of the LREC 2010 Workshop on New Challenges for NLP Frameworks*, Valletta, Malta, 2010, pp. 45–50. [Online]. Available: https://radimrehurek.com/gensim/

[4] gensim Development Team, *gensim 4.x Documentation: models.keyedvectors*, 2023. [Online]. Available: https://radimrehurek.com/gensim/models/keyedvectors.html

[5] Pygame Community, *Pygame Documentation*, 2023. [Online]. Available: https://www.pygame.org/docs/

[6] D. Turner-Evans, "Semantle," 2022. [Online]. Available: https://semantle.com/